Azure Data Factory: ForEach & Lookup Activity Demo

Scenario Overview

Scenario: Multi-Table SQL to Azure Blob Storage Export

You manage a retail database with multiple product category tables (e.g., `Electronics`, `Clothing`, `Furniture`). Instead of building a separate pipeline for each table, you will:

1. Use a **Lookup Activity** to dynamically fetch the list of table names from a control/config table stored in Azure SQL Database.
2. Use a **ForEach Activity** to iterate over each table name returned by the Lookup.
3. Inside ForEach, use a **Copy Activity** to export each table's data as a `.csv` file to an **Azure Blob Storage** container.

Why this matters: This pattern is the foundation of a *dynamic, metadata-driven pipeline* — a best practice in ADF that eliminates hardcoding and scales automatically when new tables are added.

Architecture

```text
[Azure SQL DB]                          [Azure Blob Storage]
  └─ config.TableList (control table)     └─ /exports/
  └─ dbo.Electronics                          ├─ Electronics.csv
  └─ dbo.Clothing          ──────────►        ├─ Clothing.csv
  └─ dbo.Furniture                            └─ Furniture.csv
        ▲
    Lookup reads
    table names
        │
    ForEach iterates
    + Copy Activity runs
```

Prerequisites

- An active **Azure Subscription**
- An **Azure Data Factory** instance (v2)
- An **Azure SQL Database** with:
  - A control table containing table names
  - Source data tables
- An **Azure Blob Storage** account with a container named `exports`
- Linked Services configured in ADF for both SQL DB and Blob Storage

---

Step 1: Prepare the sample tables and Control Table in Azure SQL Database

Run the following SQL script in your Azure SQL Database to create and populate the sample table:

plaintext

```
-- ============================================================
--  Retail Medallion Architecture — Source Table Scripts
--  Database  : Azure SQL Database
--  Schema    : dbo
--  Tables    : Electronics | Clothing | Furniture
--  Rows each : 10
--  Purpose   : Source tables for ADF ForEach + Lookup demo
-- ============================================================


-- ============================================================
--  1. dbo.Electronics
-- ============================================================

IF OBJECT_ID('dbo.Electronics', 'U') IS NOT NULL
    DROP TABLE dbo.Electronics;
GO

CREATE TABLE dbo.Electronics
(
    ProductId       INT             NOT NULL IDENTITY(1,1) PRIMARY KEY,
    ProductName     NVARCHAR(150)   NOT NULL,
    Brand           NVARCHAR(100)   NOT NULL,
    Category        NVARCHAR(100)   NOT NULL,
    SubCategory     NVARCHAR(100)   NOT NULL,
    SKU             NVARCHAR(50)    NOT NULL UNIQUE,
    Price           DECIMAL(10,2)   NOT NULL,
    StockQty        INT             NOT NULL DEFAULT 0,
    WarrantyMonths  INT             NOT NULL DEFAULT 12,
    IsActive        BIT             NOT NULL DEFAULT 1,
    CreatedDate     DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME(),
    ModifiedDate    DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME()
);
GO

INSERT INTO dbo.Electronics
    (ProductName, Brand, Category, SubCategory, SKU, Price, StockQty,
WarrantyMonths, IsActive)
VALUES
    ('Galaxy S25 Ultra',            'Samsung',  'Electronics', 'Smartphones',
'ELEC-SAM-001', 1299.99,  85,  24, 1),
    ('MacBook Pro 14-inch M4',      'Apple',    'Electronics', 'Laptops',
'ELEC-APL-002', 1999.00,  40,  12, 1),
    ('Sony WH-1000XM6 Headphones',  'Sony',     'Electronics', 'Audio',
'ELEC-SNY-003',  399.99, 120,  12, 1),
    ('LG OLED C4 65" TV',           'LG',       'Electronics', 'Televisions',
'ELEC-LGE-004', 1799.00,  22,  36, 1),
```

```
    ('iPad Air 13-inch M3',         'Apple',    'Electronics', 'Tablets',
'ELEC-APL-005',  899.00,  60,  12, 1),
    ('Dell XPS 15 Laptop',          'Dell',     'Electronics', 'Laptops',
'ELEC-DEL-006', 1549.99,  35,  12, 1),
    ('Bose SoundLink Max Speaker',  'Bose',     'Electronics', 'Audio',
'ELEC-BSE-007',  399.00,  75,  12, 1),
    ('Canon EOS R8 Camera',         'Canon',    'Electronics', 'Cameras',
'ELEC-CNN-008', 1299.00,  18,  12, 1),
    ('Microsoft Surface Pro 11',    'Microsoft','Electronics', 'Tablets',
'ELEC-MSF-009', 1199.99,  45,  12, 1),
    ('Anker 200W USB-C Hub',        'Anker',    'Electronics', 'Accessories',
'ELEC-ANK-010',   89.99, 300,   6, 1);
GO

-- Verify
SELECT * FROM dbo.Electronics;
GO



-- ============================================================
--  2. dbo.Clothing
-- ============================================================

IF OBJECT_ID('dbo.Clothing', 'U') IS NOT NULL
    DROP TABLE dbo.Clothing;
GO

CREATE TABLE dbo.Clothing
(
    ProductId       INT             NOT NULL IDENTITY(1,1) PRIMARY KEY,
    ProductName     NVARCHAR(150)   NOT NULL,
    Brand           NVARCHAR(100)   NOT NULL,
    Category        NVARCHAR(100)   NOT NULL,
    SubCategory     NVARCHAR(100)   NOT NULL,
    SKU             NVARCHAR(50)    NOT NULL UNIQUE,
    Gender          NVARCHAR(20)    NOT NULL,   -- Men | Women | Unisex | Kids
    Size            NVARCHAR(10)    NOT NULL,   -- XS | S | M | L | XL | XXL
    Color           NVARCHAR(50)    NOT NULL,
    Material        NVARCHAR(100)   NOT NULL,
    Price           DECIMAL(10,2)   NOT NULL,
    StockQty        INT             NOT NULL DEFAULT 0,
    IsActive        BIT             NOT NULL DEFAULT 1,
    CreatedDate     DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME(),
    ModifiedDate    DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME()
);
GO

INSERT INTO dbo.Clothing
```

```
    (ProductName, Brand, Category, SubCategory, SKU, Gender, Size, Color,
Material, Price, StockQty, IsActive)
VALUES
    ('Classic Slim Fit Chinos',         'Levi''s',       'Clothing',
'Trousers',   'CLTH-LVS-001', 'Men',    'M',   'Khaki',      '98% Cotton, 2%
Elastane',  59.99, 200, 1),
    ('Floral Wrap Midi Dress',          'Zara',          'Clothing',
'Dresses',    'CLTH-ZRA-002', 'Women',  'S',   'Blue Floral','100% Viscose',
79.99, 150, 1),
    ('Essential Crewneck Sweatshirt',   'Nike',          'Clothing', 'Tops',
'CLTH-NKE-003', 'Unisex', 'L',   'Grey',       '80% Cotton, 20% Polyester',
64.99, 180, 1),
    ('Merino Wool Blazer',              'Ralph Lauren',  'Clothing',
'Jackets',    'CLTH-RL-004',  'Men',    'XL',  'Navy',       '100% Merino
Wool',         249.00,  60, 1),
    ('High Waist Yoga Leggings',        'Lululemon',     'Clothing',
'Activewear', 'CLTH-LUL-005', 'Women',  'M',   'Black',      '75% Nylon, 25%
Lycra',      98.00, 220, 1),
    ('Puffer Down Winter Jacket',       'The North Face','Clothing',
'Jackets',    'CLTH-TNF-006', 'Unisex', 'L',   'Red',        '100% Recycled
Polyester',  229.99,  90, 1),
    ('Graphic Print T-Shirt',           'H&M',           'Clothing', 'Tops',
'CLTH-HNM-007', 'Men',    'S',   'White',      '100% Organic Cotton',
19.99, 350, 1),
    ('Kids Denim Dungarees',            'Gap',           'Clothing', 'Kids',
'CLTH-GAP-008', 'Kids',   'M',   'Indigo',     '100% Denim',
34.99, 130, 1),
    ('Cashmere Turtleneck Sweater',     'Massimo Dutti', 'Clothing',
'Knitwear',   'CLTH-MD-009',  'Women',  'XS',  'Camel',      '100% Cashmere',
189.00,  45, 1),
    ('Running Shorts with Liner',       'Adidas',        'Clothing',
'Activewear', 'CLTH-ADI-010', 'Men',    'L',   'Black',      '88% Polyester,
12% Lycra',  44.99, 275, 1);
GO

-- Verify
SELECT * FROM dbo.Clothing;
GO


-- ============================================================
--  3. dbo.Furniture
-- ============================================================

IF OBJECT_ID('dbo.Furniture', 'U') IS NOT NULL
    DROP TABLE dbo.Furniture;
GO
```

```
CREATE TABLE dbo.Furniture
(
    ProductId           INT             NOT NULL IDENTITY(1,1) PRIMARY KEY,
    ProductName         NVARCHAR(150)   NOT NULL,
    Brand               NVARCHAR(100)   NOT NULL,
    Category            NVARCHAR(100)   NOT NULL,
    SubCategory         NVARCHAR(100)   NOT NULL,
    SKU                 NVARCHAR(50)    NOT NULL UNIQUE,
    Material            NVARCHAR(100)   NOT NULL,
    Color               NVARCHAR(50)    NOT NULL,
    WidthCm             DECIMAL(8,2)    NULL,
    DepthCm             DECIMAL(8,2)    NULL,
    HeightCm            DECIMAL(8,2)    NULL,
    WeightKg            DECIMAL(8,2)    NULL,
    AssemblyRequired    BIT             NOT NULL DEFAULT 0,
    Price               DECIMAL(10,2)   NOT NULL,
    StockQty            INT             NOT NULL DEFAULT 0,
    IsActive            BIT             NOT NULL DEFAULT 1,
    CreatedDate         DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME(),
    ModifiedDate        DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME()
);
GO

INSERT INTO dbo.Furniture
    (ProductName, Brand, Category, SubCategory, SKU, Material, Color, WidthCm,
DepthCm, HeightCm, WeightKg, AssemblyRequired, Price, StockQty, IsActive)
VALUES
    ('Karlstad 3-Seater Sofa',          'IKEA',          'Furniture', 'Sofas',
'FURN-IKA-001', 'Linen Fabric',          'Beige',        230.0, 90.0,  80.0,
62.0, 1, 699.00,  30, 1),
    ('Malm Bed Frame King Size',        'IKEA',          'Furniture', 'Beds',
'FURN-IKA-002', 'Particleboard',         'White',        208.0,180.0,  38.0,
80.0, 1, 349.00,  20, 1),
    ('Eames Lounge Chair & Ottoman',    'Herman Miller', 'Furniture',
'Chairs',         'FURN-HM-003',  'Walnut Wood & Leather', 'Black Leather',
83.0, 84.0,  84.0,  43.0, 0,5695.00,   8, 1),
    ('Farmhouse Dining Table 6-Seater', 'Pottery Barn',  'Furniture', 'Dining
Tables',  'FURN-PB-004',  'Solid Oak',             'Natural Oak',  183.0,
91.0,  76.0, 110.0, 0,1299.00,  12, 1),
    ('Kallax Shelving Unit 4x4',        'IKEA',          'Furniture',
'Storage',        'FURN-IKA-005', 'Particleboard',         'White',
147.0, 39.0, 147.0,  74.0, 1, 229.00,  45, 1),
    ('Velvet Accent Armchair',          'West Elm',      'Furniture',
'Chairs',         'FURN-WE-006',  'Velvet',                'Dusty Rose',
78.0, 82.0,  84.0,  28.0, 0, 599.00,  18, 1),
    ('Solid Walnut Writing Desk',       'Article',       'Furniture', 'Desks',
'FURN-ART-007', 'Solid Walnut',          'Walnut',       140.0, 65.0,  76.0,
48.0, 1, 849.00,  15, 1),
```

```
    ('Cloud Sectional Sofa L-Shape',    'Restoration
Hardware','Furniture','Sofas',      'FURN-RH-008',  'Performance Fabric',
'Slate Grey',   310.0,175.0,  84.0, 130.0, 1,3299.00,   6, 1),
    ('Bamboo Bedside Table',            'Crate & Barrel','Furniture',
'Bedroom',        'FURN-CB-009',  'Bamboo',                'Natural',
45.0, 40.0,  58.0,   8.0, 1, 179.00,  55, 1),
    ('Industrial TV Stand 65 inch',     'Wayfair',       'Furniture', 'TV &
Media',     'FURN-WF-010',  'Metal & Reclaimed Wood', 'Black & Brown',152.0,
40.0,  55.0,  35.0, 1, 299.99,  40, 1);
GO

-- Verify
SELECT * FROM dbo.Furniture;
GO



-- ============================================================
--  4. config.TableList  (control table for ADF ForEach demo)
-- ============================================================

IF SCHEMA_ID('config') IS NULL
    EXEC ('CREATE SCHEMA config');
GO

IF OBJECT_ID('config.TableList', 'U') IS NOT NULL
    DROP TABLE config.TableList;
GO

CREATE TABLE config.TableList
(
    Id          INT             NOT NULL IDENTITY(1,1) PRIMARY KEY,
    SchemaName  NVARCHAR(50)    NOT NULL DEFAULT 'dbo',
    TableName   NVARCHAR(100)   NOT NULL,
    IsActive    BIT             NOT NULL DEFAULT 1,
    CreatedDate DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME()
);
GO

INSERT INTO config.TableList (SchemaName, TableName, IsActive)
VALUES
    ('dbo', 'Electronics', 1),
    ('dbo', 'Clothing',    1),
    ('dbo', 'Furniture',   1);
GO

-- Verify control table (used by ADF Lookup → ForEach pipeline)
SELECT * FROM config.TableList WHERE IsActive = 1;
GO
```

```
-- ============================================================
--  5. Quick summary counts
-- ============================================================

SELECT 'Electronics' AS TableName, COUNT(*) AS RowCount FROM dbo.Electronics
UNION ALL
SELECT 'Clothing',                 COUNT(*)             FROM dbo.Clothing
UNION ALL
SELECT 'Furniture',                COUNT(*)             FROM dbo.Furniture
UNION ALL
SELECT 'config.TableList',         COUNT(*)             FROM
config.TableList;
GO
```

Run the following SQL script in your Azure SQL Database to create and populate the control table:

```plaintext
-- Create the control table
CREATE TABLE config.TableList (
    Id          INT IDENTITY(1,1) PRIMARY KEY,
    TableName   NVARCHAR(100) NOT NULL,
    SchemaName  NVARCHAR(50)  NOT NULL DEFAULT 'dbo',
    IsActive    BIT           NOT NULL DEFAULT 1
);

-- Insert table names to export
INSERT INTO config.TableList (TableName, SchemaName) VALUES
('Electronics', 'dbo'),
('Clothing',    'dbo'),
('Furniture',   'dbo');
```

Step 2: Create Linked Services in ADF

2a. Azure SQL Database Linked Service

1. In ADF Studio, go to **Manage** → **Linked Services** → **+ New**
2. Select **Azure SQL Database**

3. Fill in:
    - **Name:** `LS_AzureSqlDB`
    - **Server name:** your SQL server
    - **Database name:** your database
    - **Authentication:** SQL Authentication or Managed Identity
4. Click **Test connection** → **Create**

2b. Azure Blob Storage Linked Service

1. Click **+ New** again
2. Select **Azure Blob Storage**
3. Fill in:
    - **Name:** `LS_AzureBlobStorage`
    - **Storage account:** your storage account
4. Click **Test connection** → **Create**

Step 3: Create Datasets

3a. Source Dataset — SQL Table (Parameterized)

1. Go to **Author** → **Datasets** → **+ New Dataset**
2. Select **Azure SQL Database** → **Continue**
3. Name it `DS_SQL_SourceTable`
4. Set Linked Service to `LS_AzureSqlDB`
5. Leave **Table name** blank for now — click the **Parameters** tab
6. Add a parameter:
    - **Name:** `tableName` | **Type:** `String`
    - **Name:** `schemaName` | **Type:** `String`
7. Go back to **Connection** tab → under Table, click **Add dynamic content**
8. Enter: `@{dataset().schemaName}.@{dataset().tableName}`
9. Click **OK** → **Publish All**

3b. Sink Dataset — CSV in Blob Storage (Parameterized)

1. Click **+ New Dataset** → Select **Azure Blob Storage** → **DelimitedText (CSV)**
2. Name it `DS_Blob_CSV`
3. Set Linked Service to `LS_AzureBlobStorage`
4. Set **File path** container to `exports`
5. Click the **Parameters** tab → Add parameter:
    - **Name:** `fileName` | **Type:** `String`
6. Go to **Connection** → File name → **Add dynamic content**: `@{dataset().fileName}.csv`
7. Check **First row as header** → **Publish All**

Step 4: Create the Lookup Activity

1. Go to **Author** → **Pipelines** → **+ New Pipeline**
2. Name it `PL_DynamicTableExport`
3. From the **Activities** panel, drag **Lookup** onto the canvas
4. Name it `LKP_GetTableList`
5. Click the activity → Go to **Settings** tab:
    - **Source dataset:** `DS_SQL_SourceTable`
    - Provide dataset parameter values:
        - `schemaName` = `config`
        - `tableName` = `TableList`
    - **Use query:** Select **Query**
    - **Query:**

      ```sql
      SELECT SchemaName, TableName
      FROM config.TableList
      WHERE IsActive = 1
      ```

    - ✅ **First row only: UNCHECKED** (we want ALL rows)
6. Click **OK**

Step 5: Create the ForEach Activity

1. Drag a **ForEach** activity onto the canvas
2. Name it `FE_IterateTables`
3. Draw a **Success** arrow from `LKP_GetTableList` → `FE_IterateTables`
4. Click `FE_IterateTables` → Go to **Settings** tab:
    - **Sequential:** Toggle **OFF** (run in parallel) — or ON if order matters
    - **Batch count:** `3` (max parallel iterations; adjust as needed)
    - **Items:** Click **Add dynamic content** and enter:

```text
@activity('LKP_GetTableList').output.value
```

> This passes the array of rows from Lookup into ForEach.

Step 6: Add Copy Activity Inside ForEach

1. Click the **pencil icon (✏️)** on the `FE_IterateTables` activity to open its inner canvas
2. Drag a **Copy Data** activity onto the inner canvas
3. Name it `CPY_ExportTableToBlob`

Source Tab:

- **Source dataset:** `DS_SQL_SourceTable`
- **Dataset properties:**
    - `schemaName` → Add dynamic content: `@item().SchemaName`
    - `tableName` → Add dynamic content: `@item().TableName`
- **Use query:** `Table` (reads entire table)

Sink Tab:

- **Sink dataset:** `DS_Blob_CSV`
- **Dataset properties:**
  - `fileName` → Add dynamic content: `@item().TableName`

Mapping Tab:

- Leave as **Auto mapping** (ADF will detect columns automatically)

4. Click the back arrow ← to return to the main pipeline canvas

---

Step 7: Debug and Validate the Pipeline

1. Click **Debug** in the top toolbar
2. The pipeline will execute with these steps:
   - `LKP_GetTableList` fetches 3 rows from `config.TableList`
   - `FE_IterateTables` iterates 3 times (once per table)
   - Each iteration runs `CPY_ExportTableToBlob` for its respective table
3. Monitor each activity in the **Output** panel at the bottom
4. Verify in **Azure Blob Storage** → `exports` container:
   - `Electronics.csv` ✅
   - `Clothing.csv` ✅
   - `Furniture.csv` ✅

Step 8: Add a Trigger (Optional — Production Deployment)

1. Click **Add trigger** → **New/Edit**
2. Choose a **Schedule trigger** (e.g., daily at 2:00 AM UTC)
3. Click **OK** → **Publish All**

To add a new table to the export in the future, simply insert a new row into `config.TableList` — **no pipeline changes required!**

Key Concepts Recap

Concept	What It Does in This Demo
Lookup Activity	Queries `config.TableList` and returns an array of table name rows
ForEach Activity	Iterates over each row in the Lookup output array
`@activity('...').output.value`	Extracts the result array from Lookup to feed ForEach
`@item()`	References the current iteration's object inside ForEach
Parameterized Datasets	Allows a single dataset definition to serve multiple tables dynamically
Metadata-driven pattern	Adding new tables only requires a DB insert, not a pipeline change

---

Common Pitfalls to Avoid

- ❌ **"First row only" checked on Lookup** — This returns only 1 row; always uncheck it when expecting multiple rows.
- ❌ **Hardcoding table names** — Defeats the purpose of this pattern; always use `@item()`.
- ❌ **Batch count too high** — Setting it above the SQL connection pool limit can cause throttling; keep it at 3–5.
- ❌ **Not publishing before debugging** — Always click **Publish All** after changes, or use **Save all** before a Debug run.